

Introduction

The purpose of this assignment is to use k-nearest neighbors classification to predict whether a Universal Bank customer will accept a personal loan offer. The bank wants to improve its marketing efforts by identifying liability customers who are more likely to become personal loan customers.

The dataset contains 5,000 customer records, including demographic information, banking relationship information, and whether the customer accepted a personal loan offer in a previous campaign. The target variable is Personal Loan, where 1 means the customer accepted the loan offer and 0 means the customer did not accept the loan offer.

Load Packages and Data

```
# Run this once if needed:  
# install.packages(c("caret", "class", "knitr"))
```

```
library(caret)  
library(class)  
library(knitr)
```

```
bank <- read.csv("UniversalBank.csv")
```

```
names(bank) <- make.names(names(bank))
```

```
head(bank)
```

```
##   ID Age Experience Income ZIP.Code Family CCAvg Education Mortgage  
## 1  1  25          1     49   91107      4   1.6          1         0  
## 2  2  45         19     34   90089      3   1.5          1         0  
## 3  3  39         15     11   94720      1   1.0          1         0  
## 4  4  35          9    100   94112      1   2.7          2         0  
## 5  5  35          8     45   91330      4   1.0          2         0  
## 6  6  37         13     29   92121      4   0.4          2        155  
## Personal.Loan Securities.Account CD.Account Online CreditCard  
## 1          0              1          0      0          0  
## 2          0              1          0      0          0  
## 3          0              0          0      0          0  
## 4          0              0          0      0          0  
## 5          0              0          0      0          1  
## 6          0              0          0      1          0
```

```
str(bank)
```

```
## 'data.frame':   5000 obs. of  14 variables:  
##  $ ID          : int  1 2 3 4 5 6 7 8 9 10 ...  
##  $ Age         : int  25 45 39 35 35 37 53 50 35 34 ...  
##  $ Experience  : int  1 19 15 9 8 13 27 24 10 9 ...  
##  $ Income     : int  49 34 11 100 45 29 72 22 81 180 ...  
##  $ ZIP.Code   : int  91107 90089 94720 94112 91330 92121 91711 93943 90089 93023 ...  
##  $ Family     : int  4 3 1 1 4 4 2 1 3 1 ...  
##  $ CCAvg      : num  1.6 1.5 1 2.7 1 0.4 1.5 0.3 0.6 8.9 ...
```

```
## $ Education      : int  1 1 1 2 2 2 2 3 2 3 ...
## $ Mortgage       : int  0 0 0 0 0 155 0 0 104 0 ...
## $ Personal.Loan   : int  0 0 0 0 0 0 0 0 0 1 ...
## $ Securities.Account: int  1 1 0 0 0 0 0 0 0 0 ...
## $ CD.Account      : int  0 0 0 0 0 0 0 0 0 0 ...
## $ Online          : int  0 0 0 0 0 1 1 0 1 0 ...
## $ CreditCard      : int  0 0 0 0 1 0 0 1 0 0 ...
```

Data Preparation

The ID and ZIP code fields are removed because they are identifiers rather than useful predictors. The Education variable has more than two categories, so it is converted into dummy variables.

```
bank$Education <- factor(bank$Education)

education_dummies <- model.matrix(~ Education - 1, data = bank)
colnames(education_dummies) <- c("Education_1", "Education_2", "Education_3")

bank_model <- cbind(
  bank[, !(names(bank) %in% c("ID", "ZIP.Code", "Education"))],
  education_dummies
)

bank_model$Personal.Loan <- factor(bank_model$Personal.Loan, levels = c(0, 1))

head(bank_model)
```

```
##   Age Experience Income Family CCAvg Mortgage Personal.Loan Securities.Account
## 1  25          1     49      4   1.6         0           0           1
## 2  45         19     34      3   1.5         0           0           1
## 3  39         15     11      1   1.0         0           0           0
## 4  35          9    100      1   2.7         0           0           0
## 5  35          8     45      4   1.0         0           0           0
## 6  37         13     29      4   0.4        155         0           0
##   CD.Account Online CreditCard Education_1 Education_2 Education_3
## 1          0      0           0           1           0           0
## 2          0      0           0           1           0           0
## 3          0      0           0           1           0           0
## 4          0      0           0           0           1           0
## 5          0      0           1           0           1           0
## 6          0      1           0           0           1           0
```

```
summary(bank_model$Personal.Loan)
```

```
##      0      1
## 4520  480
```

1. Partition Data into Training and Validation Sets

The data is partitioned into 60% training and 40% validation. The training set is used to build the k-NN model, while the validation set is used to evaluate model performance.

```
set.seed(123)

train_index <- createDataPartition(bank_model$Personal.Loan, p = 0.60, list = FALSE)
```

```

train_data <- bank_model[train_index, ]
validation_data <- bank_model[-train_index, ]

predictors <- setdiff(names(bank_model), "Personal.Loan")

train_x <- train_data[, predictors]
validation_x <- validation_data[, predictors]

train_y <- train_data$Personal.Loan
validation_y <- validation_data$Personal.Loan

table(train_y)

```

```

## train_y
##      0      1
## 2712  288

```

```
table(validation_y)
```

```

## validation_y
##      0      1
## 1808  192

```

Normalize the Predictor Variables

Since k-NN depends on distance calculations, predictor variables need to be normalized. This prevents variables with larger numeric ranges from dominating the distance calculation.

```

normalization_model <- preProcess(train_x, method = "range")

train_x_norm <- predict(normalization_model, train_x)
validation_x_norm <- predict(normalization_model, validation_x)

summary(train_x_norm)

```

```

##      Age      Experience      Income      Family
## Min.   :0.0000   Min.   :0.0000   Min.   :0.0000   Min.   :0.0000
## 1st Qu.:0.2727   1st Qu.:0.2826   1st Qu.:0.1435   1st Qu.:0.0000
## Median :0.5000   Median :0.5000   Median :0.2593   Median :0.3333
## Mean   :0.5050   Mean   :0.4999   Mean   :0.3048   Mean   :0.4633
## 3rd Qu.:0.7273   3rd Qu.:0.7174   3rd Qu.:0.4167   3rd Qu.:0.6667
## Max.   :1.0000   Max.   :1.0000   Max.   :1.0000   Max.   :1.0000
##      CCAvg      Mortgage      Securities.Account      CD.Account
## Min.   :0.0000   Min.   :0.00000   Min.   :0.0000   Min.   :0.00000
## 1st Qu.:0.0700   1st Qu.:0.00000   1st Qu.:0.0000   1st Qu.:0.00000
## Median :0.1600   Median :0.00000   Median :0.0000   Median :0.00000
## Mean   :0.1966   Mean   :0.09011   Mean   :0.1013   Mean   :0.06267
## 3rd Qu.:0.2600   3rd Qu.:0.16063   3rd Qu.:0.0000   3rd Qu.:0.00000
## Max.   :1.0000   Max.   :1.00000   Max.   :1.0000   Max.   :1.00000
##      Online      CreditCard      Education_1      Education_2
## Min.   :0.0000   Min.   :0.000   Min.   :0.0000   Min.   :0.0000
## 1st Qu.:0.0000   1st Qu.:0.000   1st Qu.:0.0000   1st Qu.:0.0000
## Median :1.0000   Median :0.000   Median :0.0000   Median :0.0000
## Mean   :0.5957   Mean   :0.287   Mean   :0.4183   Mean   :0.2843
## 3rd Qu.:1.0000   3rd Qu.:1.000   3rd Qu.:1.0000   3rd Qu.:1.0000

```

```
## Max.      :1.0000    Max.      :1.000    Max.      :1.0000    Max.      :1.0000
## Education_3
## Min.      :0.0000
## 1st Qu.:0.0000
## Median :0.0000
## Mean     :0.2973
## 3rd Qu.:1.0000
## Max.      :1.0000
```

New Customer Record

The following customer is classified using k-NN:

Age = 40, Experience = 10, Income = 84, Family = 2, CCAvg = 2, Education_1 = 0, Education_2 = 1, Education_3 = 0, Mortgage = 0, Securities Account = 0, CD Account = 0, Online = 1, and Credit Card = 1.

```
new_customer <- data.frame(
  Age = 40,
  Experience = 10,
  Income = 84,
  Family = 2,
  CCAvg = 2,
  Mortgage = 0,
  Securities.Account = 0,
  CD.Account = 0,
  Online = 1,
  CreditCard = 1,
  Education_1 = 0,
  Education_2 = 1,
  Education_3 = 0
)

new_customer <- new_customer[, predictors]

new_customer_norm <- predict(normalization_model, new_customer)
new_customer_norm
```

```
##           Age Experience    Income    Family CCAvg Mortgage Securities.Account
## 1 0.3863636 0.2826087 0.3518519 0.3333333    0.2         0              0
##   CD.Account Online CreditCard Education_1 Education_2 Education_3
## 1           0         1           1           0           1           0
```

k-NN Classification with $k = 1$

```
pred_new_k1 <- knn(
  train = train_x_norm,
  test = new_customer_norm,
  cl = train_y,
  k = 1,
  prob = TRUE
)

pred_new_k1
```

```
## [1] 0
```

```
## attr(,"prob")
## [1] 1
## Levels: 0 1
```

Using $k = 1$, the new customer is classified as class 0. Since class 1 represents loan acceptance and class 0 represents non-acceptance, this means the customer is predicted as not likely to accept the loan offer.

2. Choosing a Value of k

The value of k controls how many nearest neighbors are used to classify a new observation. A small value of k can overfit by following noise in the training data too closely. A large value of k can smooth too much and ignore useful local patterns in the predictor variables.

To choose k , I tested odd values of k from 1 through 25 and selected the value with the best validation accuracy.

```
k_values <- seq(1, 25, by = 2)

tuning_results <- data.frame(
  k = integer(),
  Accuracy = numeric(),
  Sensitivity = numeric(),
  Specificity = numeric()
)

for (k in k_values) {
  validation_pred <- knn(
    train = train_x_norm,
    test = validation_x_norm,
    cl = train_y,
    k = k
  )

  cm <- confusionMatrix(
    data = validation_pred,
    reference = validation_y,
    positive = "1"
  )

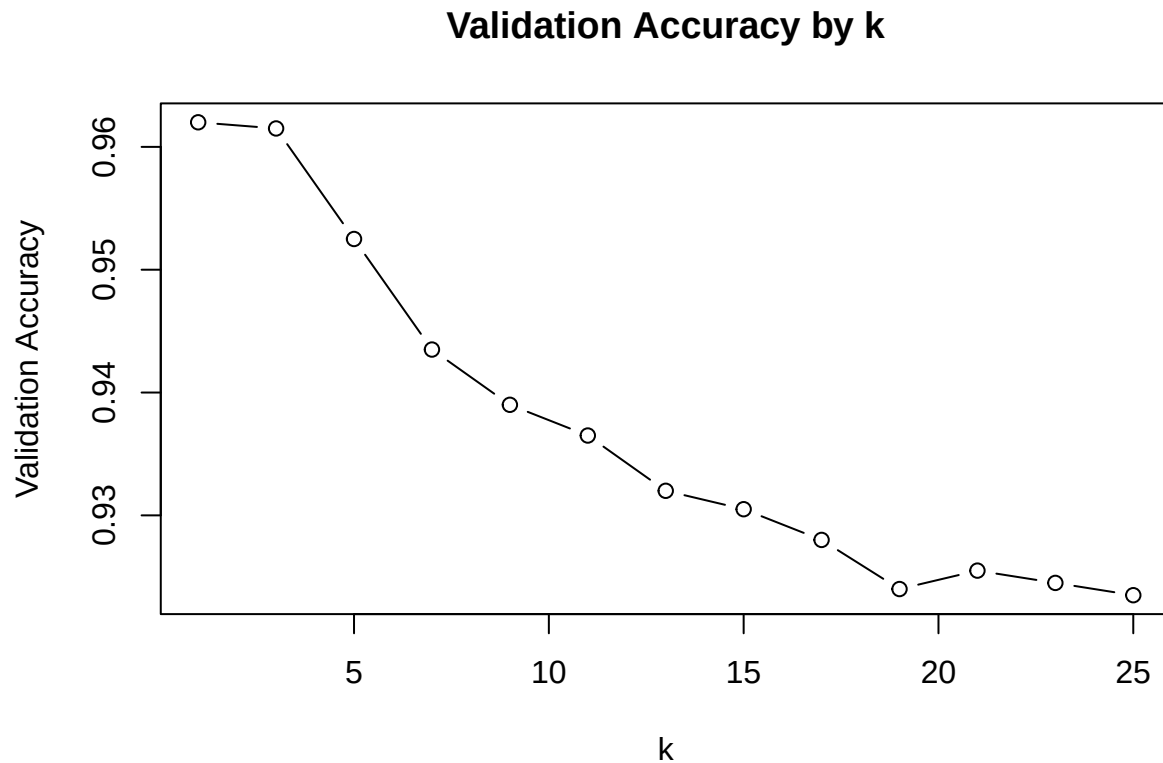
  tuning_results <- rbind(
    tuning_results,
    data.frame(
      k = k,
      Accuracy = as.numeric(cm$overall["Accuracy"]),
      Sensitivity = as.numeric(cm$byClass["Sensitivity"]),
      Specificity = as.numeric(cm$byClass["Specificity"])
    )
  )
}

kable(tuning_results, digits = 4)
```

k	Accuracy	Sensitivity	Specificity
1	0.9620	0.7135	0.9884
3	0.9615	0.6302	0.9967

k	Accuracy	Sensitivity	Specificity
5	0.9525	0.5521	0.9950
7	0.9435	0.4635	0.9945
9	0.9390	0.4167	0.9945
11	0.9365	0.4010	0.9934
13	0.9320	0.3594	0.9928
15	0.9305	0.3385	0.9934
17	0.9280	0.3229	0.9923
19	0.9240	0.2812	0.9923
21	0.9255	0.2708	0.9950
23	0.9245	0.2448	0.9967
25	0.9235	0.2240	0.9978

```
plot(
  tuning_results$k,
  tuning_results$Accuracy,
  type = "b",
  xlab = "k",
  ylab = "Validation Accuracy",
  main = "Validation Accuracy by k"
)
```



```
best_k <- tuning_results$k[which.max(tuning_results$Accuracy)]
best_k
```

```
## [1] 1
```

Based on the validation results, the selected value of k is 1. This value provides the best validation accuracy among the tested odd k values and is used for the remaining parts of the assignment.

3. Confusion Matrix for Validation Data Using the Best k

```
validation_pred_best <- knn(
  train = train_x_norm,
  test = validation_x_norm,
  cl = train_y,
  k = best_k
)

validation_cm <- confusionMatrix(
  data = validation_pred_best,
  reference = validation_y,
  positive = "1"
)

validation_cm

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 1787   55
##           1   21  137
##
##           Accuracy : 0.962
##           95% CI : (0.9527, 0.9699)
##       No Information Rate : 0.904
##       P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.7623
##
##  Mcnemar's Test P-Value : 0.0001535
##
##           Sensitivity : 0.7135
##           Specificity : 0.9884
##       Pos Pred Value : 0.8671
##       Neg Pred Value : 0.9701
##           Prevalence : 0.0960
##       Detection Rate : 0.0685
##   Detection Prevalence : 0.0790
##       Balanced Accuracy : 0.8510
##
##       'Positive' Class : 1
##
```

The confusion matrix above shows how well the k-NN model performed on the validation data using the selected value of k. The matrix compares the predicted loan acceptance results against the actual loan acceptance results.

4. Classify the New Customer Using the Best k

```
pred_new_best_k <- knn(
  train = train_x_norm,
  test = new_customer_norm,
```

```

    cl = train_y,
    k = best_k,
    prob = TRUE
)

pred_new_best_k

## [1] 0
## attr(,"prob")
## [1] 1
## Levels: 0 1

```

Using the selected value of k , the new customer is classified as class 0. Since class 1 represents loan acceptance and class 0 represents non-acceptance, this means the customer is predicted as not likely to accept the loan offer.

5. Repartition Data into Training, Validation, and Test Sets

The data is repartitioned into 50% training, 30% validation, and 20% test. The same selected value of k is then applied to the new split.

```

set.seed(123)

train2_index <- createDataPartition(bank_model$Personal.Loan, p = 0.50, list = FALSE)

train2_data <- bank_model[train2_index, ]
remaining_data <- bank_model[-train2_index, ]

validation2_index <- createDataPartition(remaining_data$Personal.Loan, p = 0.60, list = FALSE)

validation2_data <- remaining_data[validation2_index, ]
test2_data <- remaining_data[-validation2_index, ]

train2_x <- train2_data[, predictors]
validation2_x <- validation2_data[, predictors]
test2_x <- test2_data[, predictors]

train2_y <- train2_data$Personal.Loan
validation2_y <- validation2_data$Personal.Loan
test2_y <- test2_data$Personal.Loan

table(train2_y)

## train2_y
##      0      1
## 2260   240

table(validation2_y)

## validation2_y
##      0      1
## 1356   144

table(test2_y)

## test2_y
##      0      1

```



```
## 904 96
```

Normalize the New Partitions

The normalization model is fit only on the training data and then applied to the validation and test sets. This avoids using information from the validation or test sets during preprocessing.

```
normalization_model2 <- preProcess(train2_x, method = "range")

train2_x_norm <- predict(normalization_model2, train2_x)
validation2_x_norm <- predict(normalization_model2, validation2_x)
test2_x_norm <- predict(normalization_model2, test2_x)
```

Confusion Matrices for Training, Validation, and Test Sets

```
train2_pred <- knn(
  train = train2_x_norm,
  test = train2_x_norm,
  cl = train2_y,
  k = best_k
)

validation2_pred <- knn(
  train = train2_x_norm,
  test = validation2_x_norm,
  cl = train2_y,
  k = best_k
)

test2_pred <- knn(
  train = train2_x_norm,
  test = test2_x_norm,
  cl = train2_y,
  k = best_k
)

train2_cm <- confusionMatrix(train2_pred, train2_y, positive = "1")
validation2_cm <- confusionMatrix(validation2_pred, validation2_y, positive = "1")
test2_cm <- confusionMatrix(test2_pred, test2_y, positive = "1")

train2_cm
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction    0    1
##              0 2260    0
##              1    0 240
##
##              Accuracy : 1
##              95% CI : (0.9985, 1)
##              No Information Rate : 0.904
##              P-Value [Acc > NIR] : < 2.2e-16
##
```

```
##                Kappa : 1
##
## Mcnemar's Test P-Value : NA
##
##          Sensitivity : 1.000
##          Specificity : 1.000
##          Pos Pred Value : 1.000
##          Neg Pred Value : 1.000
##          Prevalence : 0.096
##          Detection Rate : 0.096
##          Detection Prevalence : 0.096
##          Balanced Accuracy : 1.000
##
##          'Positive' Class : 1
##
```

validation2_cm

```
## Confusion Matrix and Statistics
##
##          Reference
## Prediction    0    1
##          0 1335   39
##          1   21  105
##
##          Accuracy : 0.96
##          95% CI : (0.9488, 0.9693)
##          No Information Rate : 0.904
##          P-Value [Acc > NIR] : < 2e-16
##
##          Kappa : 0.7559
##
## Mcnemar's Test P-Value : 0.02819
##
##          Sensitivity : 0.7292
##          Specificity : 0.9845
##          Pos Pred Value : 0.8333
##          Neg Pred Value : 0.9716
##          Prevalence : 0.0960
##          Detection Rate : 0.0700
##          Detection Prevalence : 0.0840
##          Balanced Accuracy : 0.8568
##
##          'Positive' Class : 1
##
```

test2_cm

```
## Confusion Matrix and Statistics
##
##          Reference
## Prediction    0    1
##          0  893   33
##          1   11   63
##
##          Accuracy : 0.956
```

```
##          95% CI : (0.9414, 0.9679)
##    No Information Rate : 0.904
##    P-Value [Acc > NIR] : 5.202e-10
##
##          Kappa : 0.7176
##
##    McNemar's Test P-Value : 0.001546
##
##          Sensitivity : 0.6562
##          Specificity : 0.9878
##    Pos Pred Value : 0.8514
##    Neg Pred Value : 0.9644
##          Prevalence : 0.0960
##    Detection Rate : 0.0630
##    Detection Prevalence : 0.0740
##    Balanced Accuracy : 0.8220
##
##    'Positive' Class : 1
##
```

Comparison of Model Performance

```
comparison <- data.frame(
  Data_Set = c("Training", "Validation", "Test"),
  Accuracy = c(
    as.numeric(train2_cm$overall["Accuracy"]),
    as.numeric(validation2_cm$overall["Accuracy"]),
    as.numeric(test2_cm$overall["Accuracy"])
  ),
  Sensitivity = c(
    as.numeric(train2_cm$byClass["Sensitivity"]),
    as.numeric(validation2_cm$byClass["Sensitivity"]),
    as.numeric(test2_cm$byClass["Sensitivity"])
  ),
  Specificity = c(
    as.numeric(train2_cm$byClass["Specificity"]),
    as.numeric(validation2_cm$byClass["Specificity"]),
    as.numeric(test2_cm$byClass["Specificity"])
  )
)

kable(comparison, digits = 4)
```

Data_Set	Accuracy	Sensitivity	Specificity
Training	1.000	1.0000	1.0000
Validation	0.960	0.7292	0.9845
Test	0.956	0.6562	0.9878

The training set performance is expected to be stronger because the model is being evaluated using records from the same set used to define the neighbors. This can make training performance somewhat optimistic. The validation and test sets provide a better estimate of how the model performs on unseen data.

The validation and test confusion matrices may differ because they contain different customer records. Even

when the data is split in a balanced way, each partition is still only a sample of the full dataset. The test set is especially important because it is held out from model selection and gives the most honest estimate of final model performance. If the validation and test results are reasonably close, that suggests the model is generalizing consistently. If the training results are much stronger than the validation and test results, that may indicate overfitting.

Conclusion

The k-NN model was used to classify whether Universal Bank customers are likely to accept a personal loan offer. The data was prepared by removing identifier fields, creating dummy variables for Education, normalizing the predictors, and partitioning the data into training, validation, and test sets. The value of k was selected by comparing validation performance across multiple odd k values.

The selected k was then used to classify the provided new customer and to evaluate model performance with confusion matrices. Overall, the assignment shows how k-NN can be used for classification and how validation and test data help evaluate whether the model is likely to perform well on new customers.